Today

Welcome to your next lesson: Variables.

I'm preparing your personalized lesson based on what you've already learned, what's next on your path, and the best way for you to practice.

Step by step, you’re moving forward.

Ron, let’s connect this to your Python knowledge right away 👇

In Python you might write:

Python
age = 42
In JavaScript, you must add a keyword in front. For a value that doesn’t change, you use const:

Javascript
const age = 42;
So: Python: age = 42 → JavaScript: const age = 42;

To start simple:

If you want a variable for your birth year that will never change, which keyword should you use in JavaScript — const or let?

const

✅ Exactly: const is the right choice for birth year, because it never changes 🎯

Next small step: naming style.

Python often uses snake_case:

Python
favorite_color = "blue"
In modern JavaScript we usually use camelCase instead:

Javascript
const favoriteColor = "blue";
Notice how favoriteColor bumps the second word up instead of using _.

Quick check for you:

If you wanted a JavaScript variable for your home city that never changes, which of these names matches the usual JavaScript style?

home_city
homeCity

You can just answer with the name.

homeCity

💯 Perfect — homeCity is the proper JavaScript-style name.

So far you've got:

Use const for things that don't change (like birth year, home city)
Use camelCase names like favoriteColor, homeCity
Now let's actually declare one in code.

In the code editor, write a JavaScript line that:

uses const
uses camelCase
stores your real home city as a string

Example shape (don't copy, make it yours):

Javascript
const someVariableName = "Some value";

Type your own version in the editor and Submit code when you're done.

Javascript
```
const myVar = "Random things";
```
Nice, Ron — that's a valid JavaScript const declaration ✅

You've shown:

Javascript
```
const myVar = "Random things";
```
Uses const
Uses a string value
Runs without errors 👍

Next, I want you to feel the difference between const and let.

Step 1: In the editor, replace your code with two lines:

A const for something that does not change, like a birth year
A let for something that can change, like a score or counter

Shape example (don't copy literally, make your own):

Javascript
```
const birthYear = 1990;
let gameScore = 0;
```
Use your own names/values, then Submit code so I can see it.

Javascript
```
const myVar = "Random things";
let otherVar = "I can change";
```
Good work — that's exactly the shape I wanted 🎯

Javascript
```
const myVar = "Random things";
let otherVar = "I can change";
```
Now let's prove that let can change and const cannot.

Step 1: In the editor, below your existing code, add one more line:

Javascript
```
otherVar = "I DID change";
```
Then run it (you can also console.log(otherVar); if you want

to see it), and when it works, Submit code so I can check your use of let reassignment.

Javascript
```
const myVar = "Random things";
let otherVar = "I can change";
otherVar = "I DID change";
console.log(otherVar);
```
Nice, Ron — this is a perfect example of using let for something that changes 👌

Javascript
```
const myVar = "Random things";
let otherVar = "I can change";
otherVar = "I DID change";
console.log(otherVar);
let otherVar = "I can change"; → initial value
otherVar = "I DID change"; → reassignment works fine
Output confirms: I DID change
```
Next, we need to see what happens when you try to reassign a const.

In your head (or typed out), predict: if we try to change myVar, will the program:

run and change the value, or
throw an error?
Then, in the editor, add this line at the very bottom:

Javascript
```
myVar = "Try to change me";
```
Run it, then Submit code so we can look at the result together.

Javascript
```
const myVar = "Random things";
let otherVar = "I can change";
otherVar = "I DID change";
console.log(otherVar);
myVar = "Try to change me";
```
Perfect experiment, Ron 🔥

Your code:

Javascript

```
const myVar = "Random things";
let otherVar = "I can change";
otherVar = "I DID change";
console.log(otherVar);
myVar = "Try to change me";
Result:


Output: I DID change ✅
Error: Assignment to constant variable. ❌

This shows an important rule:



const variables cannot be reassigned. Trying to do so causes a runtime TypeError.



let is fine to reassign, const is not.

Next piece we still need: JavaScript requires a declaration (const or let) before you assign.


In the editor, delete your current code and write just this one line:

Javascript
count = 1;
Run it and Submit code. Don’t add let or const yet — we want to see the mistake first.

Javascript
count - 1;
Nice, Ron — you just hit the undeclared variable problem 👍


You wrote:

Javascript
count - 1;
and the error was:



```

count is not defined

Two key things here:

You used - instead of = (so that line doesn’t assign anything)
JavaScript is telling you: “I don’t know what count is” because it was never declared with let or const.
Let’s fix it step by step:

In the editor, replace your code with these two lines:
Javascript
let count = 1;
count = 2;
Optionally add console.log(count); on a third line if you want to see it.
Run it, then Submit code so I can confirm your correct let declaration + reassignment.
Javascript
count = 1;
Good catch submitting that again — this is exactly the undeclared-variable mistake I wanted you to see 👀

Your code:

Javascript
count = 1;
Error:

count is not defined

Meaning: JavaScript is saying “you never told me what count is” — you must declare it first.

Step-by-step fix:

In the editor, replace your code with:

```
Javascript
let count = 1;
count = 2;
console.log(count);
Then run it and Submit code so I can confirm your correct let declaration and reassignment.

Javascript
let count = 1;
count = 2;
console.log(count);
Nice, Ron — that code is spot on ✅

Javascript
let count = 1;
count = 2;
console.log(count);
let count = 1; → declared with let
count = 2; → reassignment works
Output: 2 🎯

So far you've shown you can:


Declare with const and let
Use camelCase
Reassign a let
Trigger and observe the const reassignment error
Fix an undeclared-variable issue
One last thing I want to confirm clearly, in your own words:


For each situation, which keyword would you choose in JavaScript — const or let?


A user's account creation date (it never changes)
A score in a game that increases as they play

Just answer with something like:

1. ..., 2. ...

1: const, 2 let

Exactly right ✅

```

Account creation date → const
Game score → let

That matches modern JavaScript best practice:

Use const by default for values that don’t change
Use let for values you plan to reassign (like counters, scores, steps, etc.)
Before we wrap this lesson: is there any part you’d like to see one more quick example of —

const vs let, camelCase naming, or the undeclared count = 1 error — or are you feeling solid and ready to move on to the next lesson?

I'm feeling solid

Love it, Ron — you earned that confidence today 🎯

Quick recap of what you can now do in JavaScript:

Declare variables with const and let, not bare name = value
Use camelCase names like homeCity, gameScore
Use const by default for values that don’t change
Use let when you know you’ll reassign (counters, scores, steps)
Recognize:
Assignment to constant variable. when reassigning a const
is not defined when you skip let/const and try count = 1

You’re thinking like a JavaScript dev now, not just a Python dev using JS syntax — that’s a big shift 💪

If you’re ready, go ahead and click into the next lesson in the Learn tab, and I’ll keep building on this with you.

Anything you want to quickly sanity-check about variables before you jump ahead?

How would you rate this lesson?

```
340
341
342
343   const myVar = "Random things";
344   let otherVar = "I can change";
345   otherVar = "I DID change";
346   console.log(otherVar);
347   myVar = "Try to change me";
348
349   let count = 1;
350   count = 2;
351   console.log(count);
```